JavaScript is disabled on your browser.

compact1, compact2, compact3

java.util.zip

Class DeflaterOutputStream

java.lang.Object
  java.io.OutputStream
    java.io.FilterOutputStream
      java.util.zip.DeflaterOutputStream

All Implemented Interfaces:

`Closeable`, `Flushable`, `AutoCloseable`

Direct Known Subclasses:

`GZIPOutputStream`, `ZipOutputStream`

```
public class DeflaterOutputStream
extends FilterOutputStream
```

This class implements an output stream filter for compressing data in the "deflate" compression format. It is also used as the basis for other types of compression filters, such as GZIPOutputStream.

See Also:

`Deflater`

Field Summary

Fields

Modifier and Type	Field	Description
`protected byte[]`	**`buf`**	Output buffer for writing compressed data.
`protected Deflater`	**`def`**	Compressor for this stream.

Fields inherited from class java.io.FilterOutputStream

out

Constructor Summary

Constructors

Constructor
DeflaterOutputStream(**OutputStream** out)
DeflaterOutputStream(**OutputStream** out, boolean syncFlush)
DeflaterOutputStream(**OutputStream** out, **Deflater** def)
DeflaterOutputStream(**OutputStream** out, **Deflater** def, boolean syncFlush)

```
DeflaterOutputStream(OutputStream out, Deflater def, int size)
```

```
DeflaterOutputStream(OutputStream out, Deflater def, int size, boolean syncFlush)
```

Method Summary

Concrete Methods

Modifier and Type	Method	Description
void	close()	Writes remaining compressed data to the output stream and closes the underlying stream.
protected void	deflate()	Writes next block of compressed data to the output stream.
void	finish()	Finishes writing compressed data to the output stream without closing the underlying stream.
void	flush()	Flushes the compressed output stream.
void	write(byte[] b, int off, int len)	Writes an array of bytes to the compressed output stream.
void	write(int b)	Writes a byte to the compressed output stream.

Methods inherited from class java.io.FilterOutputStream

write

Methods inherited from class java.lang.Object

clone, equals, finalize, getClass, hashCode, notify, notifyAll, toString, wait, wait, wait

Field Detail

def

```
protected Deflater def
```

Compressor for this stream.

buf

```
protected byte[] buf
```

Output buffer for writing compressed data.

Constructor Detail

DeflaterOutputStream

```
public DeflaterOutputStream(OutputStream out,
                            Deflater def,
                            int size,
                            boolean syncFlush)
```

Creates a new output stream with the specified compressor, buffer size and flush mode.

Parameters:

out - the output stream

def - the compressor ("deflater")

size - the output buffer size

syncFlush - if true the flush() method of this instance flushes the compressor with flush mode Deflater.SYNC_FLUSH before flushing the output stream, otherwise only flushes the output stream

Throws:

IllegalArgumentException - if size <= 0

Since:

1.7

DeflaterOutputStream

```
public DeflaterOutputStream(OutputStream out,
                            Deflater def,
                            int size)
```

Creates a new output stream with the specified compressor and buffer size.

The new output stream instance is created as if by invoking the 4-argument constructor DeflaterOutputStream(out, def, size, false).

Parameters:

out - the output stream

def - the compressor ("deflater")

size - the output buffer size

Throws:

IllegalArgumentException - if size <= 0

DeflaterOutputStream

```
public DeflaterOutputStream(OutputStream out,
                            Deflater def,
                            boolean syncFlush)
```

Creates a new output stream with the specified compressor, flush mode and a default buffer size.

Parameters:

out - the output stream

def - the compressor ("deflater")

syncFlush - if true the flush() method of this instance flushes the compressor with flush mode Deflater.SYNC_FLUSH before flushing the output stream, otherwise only flushes the output stream

Since:

1.7

DeflaterOutputStream

```
public DeflaterOutputStream(OutputStream out,
                            Deflater def)
```

Creates a new output stream with the specified compressor and a default buffer size.

The new output stream instance is created as if by invoking the 3-argument constructor DeflaterOutputStream(out, def, false).

Parameters:

`out - the output stream`

`def - the compressor ("deflater")`

DeflaterOutputStream

```
public DeflaterOutputStream(OutputStream out,
                            boolean syncFlush)
```

Creates a new output stream with a default compressor, a default buffer size and the specified flush mode.

Parameters:

`out - the output stream`

`syncFlush - if true the flush() method of this instance flushes the compressor with flush mode Deflater.SYNC_FLUSH before flushing the output stream, otherwise only flushes the output stream`

Since:

`1.7`

DeflaterOutputStream

```
public DeflaterOutputStream(OutputStream out)
```

Creates a new output stream with a default compressor and buffer size.

The new output stream instance is created as if by invoking the 2-argument constructor DeflaterOutputStream(out, false).

Parameters:

`out - the output stream`

Method Detail

write

```
public void write(int b)
           throws IOException
```

Writes a byte to the compressed output stream. This method will block until the byte can be written.

Overrides:

`write` in class `FilterOutputStream`

Parameters:

`b` - the byte to be written

Throws:

`IOException` - if an I/O error has occurred

write

```
public void write(byte[] b,
                  int off,
                  int len)
           throws IOException
```

Writes an array of bytes to the compressed output stream. This method will block until all the bytes are written.

Overrides:

`write` in class `FilterOutputStream`

Parameters:

`b` - the data to be written

`off` - the start offset of the data

`len` - the length of the data

Throws:

`IOException` - if an I/O error has occurred

See Also:

`FilterOutputStream.write(int)`

finish

```
public void finish()
            throws IOException
```

Finishes writing compressed data to the output stream without closing the underlying stream. Use this method when applying multiple filters in succession to the same output stream.

Throws:

`IOException` - if an I/O error has occurred

close

```
public void close()
           throws IOException
```

Writes remaining compressed data to the output stream and closes the underlying stream.

Specified by:

`close` in interface `Closeable`

Specified by:

`close` in interface `AutoCloseable`

Overrides:

`close` in class `FilterOutputStream`

Throws:

`IOException` - if an I/O error has occurred

See Also:

`FilterOutputStream.flush()`, `FilterOutputStream.out`

deflate

```
protected void deflate()
                throws IOException
```

Writes next block of compressed data to the output stream.

Throws:

`IOException` - if an I/O error has occurred

flush

```
public void flush()
           throws IOException
```

Flushes the compressed output stream. If `syncFlush` is `true` when this compressed output stream is constructed, this method first flushes the underlying `compressor` with the flush mode `Deflater.SYNC_FLUSH` to force

all pending data to be flushed out to the output stream and then flushes the output stream. Otherwise this method only flushes the output stream without flushing the `compressor`.

Specified by:

`flush` in interface `Flushable`

Overrides:

`flush` in class `FilterOutputStream`

Throws:

`IOException` - if an I/O error has occurred

Since:

1.7

See Also:

`FilterOutputStream.out`

Submit a bug or feature
For further API reference and developer documentation, see Java SE Documentation. That documentation contains more detailed, developer-targeted descriptions, with conceptual overviews, definitions of terms, workarounds, and working code examples.
Copyright © 1993, 2026, Oracle and/or its affiliates. All rights reserved. Use is subject to license terms.
Also see the documentation redistribution policy.

Scripting on this page tracks web page traffic, but does not change the content in any way.